Short Historical Perspective

16

The first electronic, programmable computers, and, therefore, the first programming languages, appeared at the end of the 1940s. Since then, many hundreds (if not thousands) of languages have been defined. In the previous chapters of this book, we have sought to identify the most important design and implementation characteristics that are common to large classes of contemporary languages.

In this final chapter, we seek to understand what were the reasons that led, in the last seventy years, to the affirmation of these characteristics and hence to the success of some languages and the disappearance of many others.

16.1 Beginnings

The first programmable electronic computers appeared in the second half of the 1940s. In recent times when considered from the perspective of the history of science. Aeons when compared to the rate of development of information technology. To realize the distance that separates us from the early history of computing, one should think that the first computers were elephantine machines (lengths greater than 10 metres, weights greater than 4 tons), so expensive that they could only be afforded by government institutes or by large agencies, and they had less processing power than that of the programmable pocket calculators that would appear in the next few decades.

There is debate about what must be considered the first computer. Of course, this primacy depends on what we intend with the term "computer". The general agreement, in computer science circles, is that a computer in the modern sense must have the following properties: (i) it is electronic and digital; (ii) it is able to perform the four elementary arithmetic operations (iii) it is programmable; (iv) it allows the

M. Gabbrielli and S. Martini, *Programming Languages: Principles and Paradigms*,
Undergraduate Topics in Computer Science,
https://doi.org/10.1007/978-3-031-34144-1_16

storage of programs and data. Under this definition, probably the first operational computer, with adequate memory, was the EDSAC, designed and developed at the University of Cambridge by Maurice Wilkes' group. It went live in 1949, and it was influenced by fundamental work by J. Mauchly and J.P. Eckert of the Moore School at the University of Pennsylvania, who used some of J. von Neumann's ideas. Their EDVAC computer was described in the design papers with all the four properties we enumerated above. However, it was not constructed until 1951.

Instead, if we admit a more general definition, then we can also consider as the pretenders to the title of first computer ASCC/MARK I and ENIAC (1946) which, in any case, remain fundamental precursors. These machines were able to execute sequences of arithmetic operations in a controlled fashion via an actual program. The program, however, could not be stored inside the machine and was expressed using elementary formalisms, sometimes also using physical switches.

ASCC/MARK I (IBM Automatic Sequence Controlled Calculator or Harvard Mark I) was constructed in 1944 by IBM with the cooperation of Harvard University with H. Aiken as principal investigator. This machine, used by the U.S. Navy for military tasks, was rudimentary to our modern eyes. Indeed, it could only carry out the work of a few tens of people and required the use of punched tapes and other external physical media to record the instructions and to transmit data to the computation devices. Think, for example, that a (decimal) constant of 23 figures was specified manually using 23 switches, each of which could occupy 10 positions corresponding to the 10 decimal figures!

ENIAC, on the other hand, was constructed in 1946 by J. Mauchly and J.P. Eckert at the Moore School of the University of Pennsylvania and initially J. von Neumann also worked on the design. ENIAC also did not have program storage and was programmed using physical devices, for example, using electrical cables to connect the different physical parts of the computer, according to input parameters. Furthermore, many consider this machine as the first real computer because, unlike ASCC/MARK I, ENIAC was many orders of magnitude faster than humans at computing and was immediately recognised as a tool for fundamental progress. Already in 1947, L.P. Tabor of the Moore School foresaw that the speed of ENIAC and computers would allow "the solution of mathematical problems until now never considered because of the enormous amount of computation required." ENIAC was used effectively for the complicated calculation of ballistic trajectories.

The limitations of these first machines were clearly due to their novelty. All the technology needed for their hardware had to be invented, and Computer Science didn't exist yet. The importance of the programming task and the development of linguistic tools to support it were not yet recognised. In the first applications (often military), the programming activity was seen as an accessory phase, so to speak, of the calculation process. Even in more general applications, such as those of EDSAC and the machines of that generation, things did not improve much: low-level machine languages were used for programming, which basically provided a description, in binary code, of the operations and calculation mechanisms of the machine itself. These *machine languages* were composed of elementary instructions (for example, instructions for adding, loading a value into a register, and so on) that could be

immediately executed by the processor. The process of coding was completely manual, there was not even the concept of a symbolic assembly code. Machine languages are also called *first-generation* languages (or 1GL).

Without reaching the edge cases of "physical" coding constants in MARK I, it is clear that the use of such languages made the writing of programs difficult. The correction of such programs, once they had reached a certain size, was impossible. At this stage, programming is mainly a technological matter, closely coupled to the details of the specific machine being programmed. It was soon realised that, to exploit the full power of the computer, it was necessary to develop adequate formalisms that were more distant from machines and closer to the user's natural language.

A first step in this direction was the introduction of *assembly languages*. We can see these languages as symbolic representations of the machine language—there is a one-to-one correspondence between machine language instructions and assembly language codes. Programs written in assembly language are translated into programs written in machine language by a program called an *assembler*. Every model of a computer has its own assembly language, so the portability of these programs is nearly impossible. Assembly languages are also called second-generation languages (or 2GL).

The true jump in quality was achieved in the 1950s, with the introduction of *high-level languages*, also called third-generation languages (or 3GL). These were designed as abstract languages which would ignore the physical characteristics of the computer and were instead suited to express algorithms in a way that was relatively easy to parse by human users. Among the first attempts in this sense were some formalisms that permitted the use of symbolic notation for arithmetic expressions. The expressions thus encoded could then be translated automatically into instructions executable by the machine. It is from these attempts that FORTRAN (FORmula TRANslation) was born in 1957.[1] FORTRAN can be considered to be the first true high-level language.

From 1957 until today, many hundreds of programming languages have been defined and implemented. It is estimated that there have been more than a hundred widely used ones. Apart from fashions, commercial reasons, and occasional circumstances, certain guidelines and evolutionary principles can be recognised in the development of high-level languages. In the remaining part of this chapter, we will therefore try to outline these principles in such a way as to provide an orientation tool for entering the modern "Babel" of programming languages. We make no claim to exhaustiveness, however, because a complete guide would require another book.

[1] The project began within IBM in 1953, the first manual defining the language was produced in 1956, and the first compiler was ready in the spring of 1957.

16.2 Factors in the Development of Languages

High-level languages have always been designed with the aim of facilitating computer programming. However, from the 1950s to the present day, the importance and cost of the various components involved in the realization of programs have changed substantially, and thus also the priorities in language design have completely changed.

In the 1950s, hardware was certainly the most expensive and important resource—Thomas Watson, president of IBM, asserted in 1943 that there would be no market for more than five computers in the world! The first high-level languages were thus designed with the aim of obtaining efficient programs that could make the most of the hardware's potential. This attitude of the first languages is reflected in the presence of many constructs inspired directly by the structure of the physical machine. For example, the "three-way jump" present in FORTRAN was derived directly from the corresponding instruction of the IBM 704. The fact that programming was difficult and time-consuming was considered a minor problem that could be solved with more human resources, which were certainly cheaper than hardware.

Today, the situation is diametrically opposed: computer hardware is relatively cheap and efficient, and the preponderant costs in setting up a computer system are linked to the employment of computer specialists. Moreover, given the increasingly critical applications of computing systems (think of avionics applications or control systems for a nuclear power plant), there are concerns of correctness and security that fifty years ago were minimal, if not absent. Thus, modern languages are designed taking into account first the improvement of various software project activities, including the verification of the correctness of programs and their maintenance, while concerns regarding the efficient use of the physical machine has receded to the background, except in some particular cases.

Obviously, we have not gone from the 1950s vision to the current one with a seamless solution: the development of programming languages has followed a long, continuous process governed by various factors. Here we see some of the most important ones:

Hardware The type and performance of available hardware devices clearly influenced the languages that used them. We see this point better in the following sections, where we will consider, in different historical contexts, the influence of physical machines on the languages of their time.

Applications Applications of computers, initially of a purely numerical nature, have rapidly expanded to many different fields, including some in which the processing of non-numerical information is required. New fields of application may require languages with specific characteristics. For example, in the fields of artificial intelligence and knowledge management, languages are needed that allow the manipulation of symbolic formalisms rather than the solution of mathematical problems. Or, computer games are usually realised using particular programming languages.

New Methodologies The development of new programming methodologies, especially for programming on a large scale, has influenced the development of new languages. A significant example in this respect is object-oriented programming.

Implementation The implementation of the constructs of a language is significant for the development of subsequent languages as it allows one to realise the validity of a construct and its practical feasibility.

Theory Finally, we should not forget the role played by theoretical studies. They had an important part in selecting certain types of constructs and, above all, in identifying new technical tools to improve programming activities. Think, for instance, of what we have seen with regard to the elimination of the `goto` and the introduction of advanced type systems.

These factors, and others, are decisive in the life cycle of a programming language that, in most cases, is quite short. The exceptions are some languages that, due to the goodness of the initial project or, more often, for commercial and social reasons, survive for more than a decade. We will see some of these in the following.

16.3 1950s and 1960s

As we saw at the beginning of this chapter, the first computers, from the late 1940s to the early 1950s, can be considered precursors: interesting from a historical point of view, but very far removed from modern computers, both in terms of the languages used and the applications that could be implemented on them.

Towards the end of the 1950s and in the 1960s, mainframes, the first true general-purpose computers that could be used for different applications, came to the fore. They were, however, machines available only at a few major processing centres, as they had enormous size and cost (they filled a room and cost millions of Euros, in today's terms) and were only usable by specialised personnel. The IBM 360 is a famous example of a mainframe, which lasted for many years in the most important computer centres.

The processing methods used by mainframes were called *batch* processing. In this kind of system, programs were executed in a strictly sequential manner. The entire computational resource of the computer was assigned to a program which took a "batch" of data as input and produced, as output, another batch of data. When a program terminated, the next program would execute. Data and programs were initially represented on punched cards which were read by suitable equipment. The data structures mainly used for data input and output were files. Such systems did not engage in much interaction with users. For example, in the event of errors during execution, the program had to be able to restore the correct state by itself, since no external interactive intervention was possible.

The first high-level languages were developed for this type of processing system. They were languages that offer few opportunities for interaction with the machine

and which were suited to writing monolithic programs to be executed from start to finish without external interaction.

FORTRAN

As we have already mentioned, the first truly imperative high-level language can be considered to be FORTRAN, developed by J. Backus' group in 1957 and designed for numerical-scientific applications. At a time when programming was solely in assembly language and where the major concern was for program efficiency, high-level language design could not ignore the performance of compiled code. Consequently, the design of FORTRAN put performance first, and so the characteristics of a particular physical reference machine (the IBM 704) were taken into account in the design. However, unlike previous languages, FORTRAN was already in its first version a high-level language in the modern sense. In fact, this first version contained many constructs that were independent of a specific machine. FORTRAN was the first programming language to allow the direct use of symbolic, complex arithmetic expressions. After some modifications and new versions (in particular, those in 1966, 1977, and 1990), FORTRAN has survived until today and is still used for some numerical applications. However, it is an outdated language whose survival is mainly linked to practical issues, such as the existence of an extensive library of functions for scientific and engineering calculations. A FORTRAN program consists of a main routine and a number of subroutines that can be compiled separately. It is not possible to define nested environments (nested subprograms are allowed only from FORTRAN90) and there are two only kinds of environment: the local and the global. This, as we know, greatly simplifies the handling of names and the environment. Memory, either for subprograms or for the main routine, is statically allocated and there is no dynamic memory management (FORTRAN90 is again an exception by providing dynamic memory). The sequence control commands in the first version referred directly to assembly language and, therefore, `goto` was prominently used. In successive versions, more structured commands were introduced. Parameter passing is by reference. Types are present in a limited way, including only numeric (integer, real in single and double precision, complex), boolean, array, string, and file.

ALGOL

ALGOL means, more than a language, a family of imperative languages, introduced since the late 1950s. These languages, although they never achieved real commercial success, were dominant in academia from the 1960s until the 1970s and had a formidable impact on the design of all subsequent programming languages. Many of the concepts and constructs that are found in modern languages were introduced, or experimented with, for the first time, in the languages of the ALGOL family.

The progenitor was ALGOL 58, designed in 1958 by a committee led by Peter Naur. The committee was the fusion of two previous groups, one European and one American, each tasked with the definition of a new language. The name ALGOL is an acronym for ALGOrithmic Language and clearly indicates the purpose of the

language. Unlike FORTRAN, indeed, ALGOL was designed as a universal language, suited to expressing algorithms in general, rather than for use in specific types of applications. There were various revisions to the language, all originating from the collective work of an international committee (which had a turbulent life, particularly towards the end). In 1960, ALGOL 60 was defined. This is the version that is perhaps the most important (it had a minor revision in 1962). Beginning in 1966, dissenting from the majority of the committee, C.A.R. Hoare and N. Wirth started the basic design of ALGOL W, later implemented in 1968 by Wirth and which constitutes the progenitor of Pascal. The majority of the committee, instead, continued their own work which led to the definition of ALGOL 68.

ALGOL 58 and, in particular, ALGOL 60 greatly increased the machine-independence of the language, making the notation used for programming closer to mathematical notation and avoiding almost every reference to specific architectures, even at the cost of some additional design complications. For example, the input and output operations, rather than being coded by suitable instructions in the language, must be implemented by external procedures, defined for the devices of a specific installation.

ALGOL 60 made at least three fundamental contributions to modern languages. The first is parameter passing by name (see Sect. 7.1.2), a mechanism complicated to implement (and for this reason later abandoned for more then twenty years), but which has been of paramount importance in defining mechanisms (i.e., closures) for passing functions as parameters.

Another important innovation in ALGOL 60 was the introduction of blocks and, therefore, the ability to hierarchically structure the environment.

At the syntactic level, thanks to the contribution of John Backus, ALGOL 60 was the first language to use Chomsky's generative grammars to express the syntax of the language (in particular, context-free grammars were used in the form that we now know as Backus Naur Form or BNF). This novelty opened the way to a whole new area of research that has been most important to the theory and practice of compilers.

Among the other contributions of the languages in the ALGOL family to modern languages, let us further note: recursion and dynamic memory management (but for these features see also what we say below about LISP); type systems with the ability to permit new user-defined types; finally, many structured commands for sequence control in the form that we use today, (`if then else`, `for` and `while` from ALGOL 60 or `case` from ALGOL 68).

LISP

LISP (LISt Processor) was designed in 1960 by a group led by John McCarthy at MIT (Massachusetts Institute of Technology) and was one of the first languages designed for non-numeric applications. As we have already seen in the box on Sect. 6.1.1, it is a language designed to manipulate symbolic expressions (called s-expressions) to be typically used in Artificial Intelligence. Among LISP applications were the first attempts at automatic translation.

LISP was never standardized—developed and implemented over 30 years in different versions, it was never much used commercially. However, it is an important

language that experimented with several techniques that have been later used in other languages. In the academic world, LISP still enjoys a following (the Scheme language, born as a variant of LISP, was and still is used in several academic courses). The first implementations of LISP were very inefficient, so much so that architectures specially designed for LISP (so-called LISP machines) were constructed. Later, the use of various tricks, and in particular the improvement of garbage-collection techniques, allowed efficient implementations even on traditional architectures.

LISP is a functional language that, as has been said, manipulates special data structures. Every program consists of a sequence of expressions to be evaluated. Some of these expressions can be function definitions that are to be used in other expressions. Typically, the language is implemented using an interpreter and programs are evaluated in an interactive environment. Among the contributions of LISP, we can note: the introduction of higher-order programming, i.e., the possibility of constructing functions that accept as parameters and/or produce as the result of the evaluation of other functions; the dynamic management of memory using a heap; automatic garbage collection. Other specific properties of LISP have remained confined to this language, such as, for example, the use of the dynamic scope rule implemented using A-lists.

COBOL

This too, like LISP, is a language that dates back to the 1960s and here too the name is an acronym: COBOL stands for COmmon Business Oriented Language. The similarities between the two languages, however, end there. COBOL was in fact designed with the aim of obtaining a specific language for business applications whose syntax was as close as possible to that of the English language. After several revisions of the initial language, which was designed by a group led by G. Hopper at the US Department of Defence, the language became a standard in 1968, and, although in revised and modified versions, is still in use today.

COBOL programs are composed of 4 "divisions". The "procedure division" contains the code for the algorithmic aspects of the program. In the "data division" we find the descriptions of the data. The "environment division" encloses the specification of the environment external to the program provided by the physical machine on which the language is implemented. Finally, there is the "identification division", which contains information used to identify the program (name, author, etc.). The purpose of this organisation is both to separate, albeit in a crude way, the data from the programs that use it, and to separate the machine-dependent aspects from the independent ones. They are rudimentary mechanisms, surpassed by the linguistic tools we have seen exist in modern languages (types, abstract data types, objects, modules, intermediate code, etc.). Moreover, this program structure, together with the syntax close to natural language, makes even the simplest programs quite long. Memory management is entirely static.

Simula

Simula, another descendent of ALGOL 60, is a textbook case of a technology that was too advanced for its time. Developed from 1962 at the Norwegian Computing Centre by K. Nygaard and O.J. Dahl, Simula is an extension of ALGOL. It was

designed for discrete-event simulation applications, that is, for the implementation of programs that simulate load and queue situations so that fundamental parameters can be measured (average waiting time, length of the queue, etc.). In its most important version, Simula67, the language introduced the concepts of class, object, subtype, and dynamic method dispatch.[2] Simula67 was without a doubt the first object-oriented language, and it had a considerable influence on its successors (Smalltalk and C++), even if the anthropomorphic metaphor of objects exchanging messages through methods would only arrive with Smalltalk and its volcanic creator, Alan Kay.

From the linguistic viewpoint, Simula67 makes small modifications to the constructs that were already present in ALGOL 60 (the most important of which is the default mode for passing parameters which is changed from name to value-result) but adds various mechanisms, among which call-by-reference, pointers, coroutines (a mechanism for defining concurrent procedures that is fairly close to the modern concept of thread, or of generators), classes and objects. A class in Simula is a procedure that, when it terminates, leaves its activation record on the stack and returns a pointer to it. An activation record of this kind, which contains the variables declared local to the procedure (today we would say: instance variables) and (pointers to) local functions (methods), is an object. Subtypes, dynamic dispatch, and inheritance are already present in Simula67, while later versions introduced other abstraction mechanisms.

Simula has always had a big impact even outside the academic world, e.g., it is possible to find applications written in this language more than 40 years after its release.

The Birth of Concurrency

As we mentioned in Sect. 14.2, the 1960s also saw the birth of the first constructs for concurrent programming. These were low-level constructs, employed in operating systems, needed to handle the concurrency introduced into computation by device controllers and their interrupt-based communication mechanisms. From a theoretical point of view, in these years, the first synchronisation problems began to be studied, trying to identify conditions and constructs that would guarantee the independence of the execution of parallel processes (A. Bernstein) and mutual exclusion (E.W. Dijkstra). As already mentioned, semaphores were introduced in these years (they were already present in ALGOL 68). The classic problem of synchronisation of the dining philosophers also dates back to this period.

16.4 1970s

Thanks to the advent of the microprocessor, the 1970s saw the rise of the minicomputer, computers of smaller size and power comparable to that of the older mainframes.

[2] Methods are called *virtual procedures* in Simula.

From a software point of view, batch processing gave way to a more interactive approach where the user could interact directly with the execution of the program via a terminal. The characteristics of the languages developed in the mainframe era were not suited to interactive systems. For example, it became necessary to express more sophisticated operations for input and output (that were previously limited to reading and writing files). In interactive systems, moreover, there are constraints on the response time. Thus, new languages included linguistic constructs of a temporal nature (for example, timeout mechanisms). In general, "traditional" high-level languages from the 1970s allow more direct interaction with the machine than was possible with the languages of the previous generation.

By "traditional", above, we mean the imperative languages inspired by the classical computational model based on the modification of values stored in memory locations. The 1970s, however, saw the birth of two new programming paradigms: object-oriented programming and declarative programming, the latter in the two flavours of functional and logic programming. Below we will see some of the most common languages of these years.

C

Among the new languages of the 1970s, the most important is probably C, a language designed by Dennis Ritchie and Ken Thompson at AT&T Bell Laboratories. Originally designed as a systems programming language for the Unix operating system, C soon became a general-purpose language, although systems programming remains one of its more important applications. By way of an anecdote, its name derives from the fact that in 1972 it was the successor language to the B language, which in turn was a lightweight version of the BCPL system language.

Compared with the languages in the ALGOL family, from which it inherited many characteristics, C offers more opportunities to access the low-level functionality of the machine and to program interactive systems. In C, it is possible to have direct access to the character input from a terminal. Or it is possible to use specific commands to process data in real time. C rapidly established itself, both for system programming and as a language for general use, thanks to these characteristics, its compact syntax, and the possibility of compiling its programs into efficient machine code.

In the previous chapters, we saw numerous references to C for specific constructs or implementation techniques. Let us recall just some of these. We have seen how C includes many assignment operators and, especially, how the block structure is simpler than that of the languages of the ALGOL family (basically, C does not allow nested functions). This supports a much simpler handling of environments and, therefore, of the passing of functions as parameters. C has pointers that can be directly manipulated and, moreover, allows to use arrays through pointers and viceversa. On one hand, these characteristics allow for powerful and efficient operations. On the other, they may cause insidious errors. Together with the lack of a strong type system, they are one of the critical points of the language. Indeed, when one wants more reliability than efficiency, one can find valid alternatives in other modern languages.

Pascal

Pascal was developed around 1970 by Niklaus Wirth as a development and simplification of ALGOL W and was the most used educational language right up to the early 1990s. The name is in honour of the mathematician (as well as physicist, philosopher, and writer) Blaise Pascal—who, to help a father overloaded by a difficult job as part of the administration of Normandy, in 1642 designed an "arithmetic machine", the precursor of mechanical calculating machines.

One of the main reasons for the success and fame of Pascal is that it was the first language that, preceding Java and its bytecodes by nearly 20 years, introduced the concept of intermediate code as an instrument for program portability. A Pascal program was translated by the Pascal compiler (which was also written in Pascal) into P-code. P-code was a language for an intermediate machine with a stack architecture, which was then implemented in an interpretative way on the host machine. In this way, to port Pascal to a different machine, it was only necessary to rewrite the P-code interpreter. Pascal also had a fully compiled implementation, which did not use an intermediate machine and allowed for greater efficiency.

Here we list some of the more important properties of Pascal, in an attempt to compare them with C.

Pascal is a block-structured language where functions and blocks that can be nested with arbitrary complexity. This on the one hand increases the structuring of code; on the other hand, it complicates the handling of the environment and the mechanisms for passing functions as parameters. Pascal (like C), uses the static scope rule and includes dynamic memory management both using a stack (for activation records) and a heap (for explicitly allocated memory). The Pascal type system is fairly extensive and supports abstraction mechanisms by allowing the user to define new types using the `type` primitive. The types are mostly checked statically by the compiler, even if some checks may be performed only at runtime. Pascal has explicit pointers, albeit without the dangerous pointer arithmetic that C allows. Moreover, arrays and pointers are different types, thus limiting the danger of pointer manipulation. In its search for a reliable type system, Pascal is perhaps too restrictive as far as arrays are concerned. In fact, two arrays containing values of the same type but of different dimensions are of two different types, a property that makes the design of array manipulating procedures difficult. Another limitation of the language, at least in its original version, is the lack of separately compilable modules, although this has been solved in many subsequent implementations by defining external procedures.

Smalltalk

A strong limitation of Pascal, like all "conventional" programming languages from the 1970s, was the lack of mechanisms to support encapsulation and information hiding in an effective manner. Pascal supports the definition of new data types, but there is no way to couple these new types with operations that should manipulate them, so that abstraction over data could be guaranteed.

Smalltalk presents a novel way to integrate mechanisms for encapsulation and information hiding using the concepts of class and object (previously introduced by

Simula) and precise visibility rules for classes (methods are public, instance variables are private). Developed during the 1970s by Alan Kay and his group at Xerox PARC,[3] Smalltalk is a unique language. We saw some of its characteristics in Chap. 10, but their complete description would require much more space than what is available here. Unlike some object-oriented languages that were introduced later (for example, C++), Smalltalk was designed from the start to include as primitive the concept of objects rather than grafting it onto an existing language. This means that all the mechanisms used in its implementation (procedure call, memory management, etc.) were developed using this concept. Moreover, Smalltalk was designed not only to be a language but also as a sort of "total system", which included language, programming environment, and also a special dedicated machine for increased efficiency of program execution. Indeed, given that the language's type system is entirely dynamic, the implementation of an efficient method lookup mechanism was quite difficult. Very soon, however, implementations of Smalltalk were also proposed for conventional machines, with satisfactory results. A standard has never been defined for Smalltalk and various, quite different, versions of the language coexisted for a long time.

Declarative Languages

We introduced in Chap. 6 the difference between imperative and declarative programming. The motto of declarative programming is that the activity of programming should concentrate on what needs to be done, leaving the language interpreter to concentrate on how to achieve the desired result. Imperative programming, on the other hand, requires the programmer to specify both the what and the how. This is, of course, an ideal vision. Leaving to the interpreter to decide how to do the computation (that is, essentially, memory management and flow control), without the programmer providing any guidance in this sense, may impose a large penalty on program efficiency. In reality, alongside the "pure" version of declarative languages that corresponds to this vision, we find "impure" versions that add constructs of an imperative nature to improve the efficiency of programs and also to allow the use of more traditional commands (e.g. assignments).

Declarative languages can be divided into functional and logic programming languages. We will look at the two main representatives of the two classes, both of which were introduced in the 1970s.

ML

ML was born as Meta Language (hence its name) for a semi-automatic system for proving properties of programs and was developed by Robin Milner's group

[3] The Palo Alto Research Center of Xerox Corp. was a mythical research centre in those years. In addition to Smalltalk, the following were all PARC innovations: Ethernet; laser-printer technology (later developed by Adobe, a PARC spin-off); PostScript; the first personal computer (the Alto) for "office automation", equipped with a graphical user interface using the desktop metaphor; the remote procedure call.

at Edinburgh, starting in the mid-1970s. Very soon, it became a true programming language that could be used for the manipulation of symbolic information.

In ML, as in LISP, a program consists of a set of function definitions. Various imperative constructs were added to the purely functional part of ML, in particular the assignment (limited to so-called "reference cells", sort of modifiable containers bound to names in a reference model of variables.)

One of the most important contributions of ML concerned types. The language was indeed provided with a safe type, static system, which in various ways extended the types of other languages of the same period. First, the concept of type safety has a rigorous and precise definition. The system excludes (in a provable way) the possibility of unsignalled runtime errors, deriving from type violations. The ML type checker statically determines the type of every expression in a program. The language guarantees that if the type checker determines that an expression has a certain type `T`, then every evaluation of that expression will yield a value of type `T`.

Moreover, the ML type system supports a type inference mechanism. The programmer can leave some information about types unspecified, and the system will use some form of logical inference to infer the type of an identifier from how it is used. Similar mechanisms were previously studied in the context of the λ-calculus: ML was the first programming language to be equipped with such an automatic type-inference mechanism. Finally, ML supports parametric polymorphism.

PROLOG

PROLOG was defined in the 1970s as well. This was the first logic-programming language and is still available today in various versions and implementations.

If some ideas on logic programming can be traced back to the work of Kurt Gödel and Jacques Herbrand, the first solid theoretical bases were published by Alan Robinson who, in the 1960s, made an essential contribution to the theory of automatic deduction. Robinson provided a formal definition of the unification algorithm and defined *resolution*, a deduction mechanism that uses unification and that supports the proof of theorems in first-order logic.

Given its simplicity, resolution is perfect for implementing automatic theorem provers (for first-order logic), but it does not provide a computational mechanism such as the one normally provided by a programming language. The proof of a theorem does not produce an "observable" result that could be seen as the result of a computation. To obtain this computational vision of the activity of proof, 10 years had to pass and a restricted version of resolution had to be developed. This version is SLD resolution, proposed by Robert Kowalski in 1974. SLD resolution unlike previous mechanisms for automated theorem proving, allows one to prove a formula by explicitly computing the values of the variables that make the formula itself true. Therefore, at the end of the proof, those values constitute the result of the computation. If Kowalski defined the theoretical model, the PROLOG language was developed by Pierre Roussel and Alain Colmerauer who, in 1970, were working on a formalism for manipulating natural language, exploiting automatic theorem-proving mechanisms. These experiments led to the first implementation of PROLOG in 1972 and then, after various interactions with Robert Kowalski, to the 1973 version which

is mostly the same as the current versions. The ISO PROLOG standard was defined in the 1990s.

The First Concurrent Languages

The 1970s saw the development of the first concurrent languages, both in a theoretical and practical sense. Particularly important from the theoretical point of view was the introduction of so-called process calculi or process algebras.[4] These formalisms allow concurrent systems to be modelled with mathematical precision, using a few primitive constructs to express interaction, communication (typically by message exchange), and process synchronisation. Associated with these calculi, we have algebraic laws and formal semantics that allow various properties of the computations to be verified. The first, fundamental contributions in this field were those of Robin Milner, who introduced the Calculus of Communicating Systems (CCS), in the second half of the 1970s, and those of Tony Hoare, who, in 1978, published his work on Communicating Sequential Processes (CSP), a calculus of processes that later developed into the *Occam* programming language. These years also saw the first important theoretical contributions to the proof of properties such as partial correctness, mutual exclusion, and deadlock-free concurrent systems. Important in this respect were the works of Susan Owicki, David Gries, and Leslie Lamport.

Monitors developed in the 1970s mainly thanks to a famous article by Hoare in 1974, although the initial idea of encapsulating data to control access to shared variables in a concurrent environment is attributed to Edsger Dijkstra (1971) and Per Brinch Hansen (1972). The latter introduced *Concurrent Pascal* in 1975, the first concurrent language to adopt monitors. Brinch Hansen was also one of the first to come up with the idea of RPC (remote procedure call). Several other languages of this period used monitors, among them *Modula*, developed by Niklaus Wirth and *CSP/k*, developed by Ric Holt.

Finally, the 1970s were also the years of major development of the UNIX operating system, which included system calls to handle concurrency such as `fork`, `wait`, and `exit`.

16.5 1980s

The 1980s were dominated by the development of the personal computer (PC). The first commercial PC can be considered to be the Apple II, produced by Apple in 1978. Today, with the massive increase in the use of computing devices in everyday life, the PC seems an indispensable tool, but when it first appeared, most people remained sceptical about its potential. Even in 1977, Ken Olson, the president of Digital Equipment Corp. (a major producer of minicomputers at that time), stated

[4] The latter terminology, however, was introduced later, in the 1980s.

Ada Byron Lovelace

The name of the Ada language is a tribute to Ada Byron, Countess of Lovelace (1815–1852). Daughter of the poet George Byron, she was one of the first female figures in the history of automatic computing. She was a great supporter of Charles Babbage (1792–1871), a mathematician at the University of Cambridge and modern calculating machine pioneer. Babbage designed two calculating machines (the "analytic" and the "difference" engines) which were well ahead of their time and were of such technical complexity that they could not be built. In 1842, the Italian mathematician L. Menabrea published a memoir in French on Babbage's analytic engine. In 1843, Ada Lovelace translated it into English, adding several notes, where we can find a farseeing account of a calculating machine for general use.

that he did not see any reason for anyone to want to have a computer in their home! This initial confusion evaporated when, in 1981, IBM launched its first PC and Lotus created the first electronic spreadsheet. This application made people immediately see the possibilities of the new technology, which went into general use in 1984. In that year, Apple released onto the market the Macintosh—for the first time a computer for the general public had an operating system with a graphical interface based on windows, icons, and a mouse—a system similar to the windowing interface that we still use today. Later (in the 1990s), Microsoft also introduced its own Windows system.

The role of programming languages has been completely transformed by the PC. The development of systems for personal use, which provide easy-to-use graphical interfaces, brought the need to develop easily interactive graphical systems to manage windowed interfaces. These systems are produced using large and complex programs, which makes it essential to reuse already existing code (possibly produced by others). Thus, these platforms provided an application area that is ideal for object-oriented languages which provide natural mechanisms for code reuse and for organising vast and complex software projects. Indeed, the languages of the 1980s were mostly conceived as object-oriented languages, which saw significant development during this period and their first large commercial applications.

In this decade, the first embedded systems were also developed. These are systems composed of computers connected to physical devices which perform control tasks (for example, motors, parts of industrial machinery, domestic electrical goods, etc.). Embedded systems pose many problems, most of all relating to the reliability and correctness of programs. For a program that controls the engines of an aircraft, an error must not be handled interactively by the programmer and termination of the program is not an acceptable option. Moreover, embedded systems introduce other problems as far as response time is concerned. These problems are tackled by so-called real-time programming languages.

C++

The first version of C++ was defined in 1986 by Bjarne Stroustrup at AT&T's Bell Laboratories, after years of work (and after the definition of several other languages) devoted to figuring out how to add classes and inheritance to the C language without sacrificing efficiency and without compromising compatibility with existing C programs. C had to remain a subset of C++, so that any legal C program should be accepted and translated by the C++ compiler. To these primary objectives, the improvement of C's type system was added.

These objectives were substantially achieved. Even if there are some inconsistencies between C++ and C, most C programs are accepted by a C++ compiler. There was significant effort to obtain a language in which those constructs that are not used by a program do not exert a negative influence on the efficiency of the program itself. This means, in particular, that the C subset of C++ must not be affected in any way by the presence of objects and the structures required to handle them. C++ does not use any form of garbage collector, so it remains compatible with C, and it is still as efficient.

The static type system was improved and, in C++, it is possible to use a generic form of class, called a template, which supports a form of parametric polymorphism. An important design decision was that of handling C++ objects as a generalisation of the structures (`struct`) in C. As a consequence, C++ objects can be allocated in activation records on the stack and, unlike Simula (and Java), they can be manipulated directly and not only through pointers. An assignment in C++ can then copy an object to the memory space that was occupied by another object, rather than just changing a pointer (a reference).

The method lookup mechanism in C++ is simpler and more efficient than that in Smalltalk, given that C++ may use information provided by the static type system (which does exist in Smalltalk).

Finally, C++ has multiple inheritance, with all its implementation challenges. The standard version of C++ was approved in 1996.

Ada

Another important language defined in the 1980s is Ada, whose project was sponsored by the US Department of Defense. The language was defined in a rather unusual way, starting with a call for proposals from the Department of Defence, which set out the design requirements and was open to several research groups, both academic and industrial. The call was won by Jean Ichbiah in 1979 with a language based on Pascal which included many new constructs for programming real-time and embedded systems, as well as other kinds of systems. Ichbiah's proposal included abstract data types, tasks, timing mechanisms, and mechanisms for the concurrent execution of tasks. This last feature introduced topics that were new to the commercial programming languages of the time. Task is a simple concept: when task A calls task B, task A continues to be executed while B executes (while in the case of standard subprograms the execution of A is suspended while B executes). However, we already know that the concurrent execution of two tasks poses problems of synchronisation, com-

munication, and management that require linguistic constructs and implementation mechanisms (e.g. rendezvous, introduced and extensively used in this language).

The standard version of Ada was defined in 1983, although the first compilers did not appear until 1986 due to conformance testing against the standard—perhaps a unique occurrence in the history of programming languages.

The Developments of Concurrent Languages

In addition to Ada, the 1980s saw the emergence of other languages for concurrent programming. Based on the CSP theoretical model the *Occam* language was developed at INMOS, first for programming transputer[5] networks and later also for other platforms. It is a language whose guiding principle is simplicity, as its name suggests.[6] Occam was the first language to explicitly use channels. In these years, in addition to the transputer, numerous parallel architectures were developed, such as Intel's hypercube architecture machines, the Connection Machine, and Cray Research's machines. Consequently, specific programming methodologies and languages were also developed, although many of them were short-lived.

The term "process algebra" entered the literature when Jan Bergstra and Jan Willem Klop developed their Algebra of Communicating Processes (ACP).

At the beginning of the 1980s, Gregory R. Andrews developed Synchronising Resources (SR), a language designed for concurrent programming that had a certain following in the academic world. Also from these years is Linda, an alternative computation model, defined by Davide Gelernter, in which communication and synchronisation take place via a structure shared by processes called the blackboard.

Various concurrent extensions to functional and logic languages were also defined, to achieve concurrent formalisms that retained the positive aspects of declarative programming. Concurrent logic languages defined in this period include PARLOG, Concurrent Prolog, and Guarded Horn Clauses (GHC). The latter plays a special role as it was one of the key players in the Japanese FGCS (Fifth Generation Computer Systems) project. The project had colossal funding from the Japanese government (a total of about 400 million US dollars at 1992 values) and was supposed to lead to the realisation of a new generation of parallel computers, with superior performance due to innovative architectures using GHC as machine language. In fact, despite the many important scientific results obtained, the project was not a success and the machines built were soon outperformed by other parallel computers.

[5] The transputer is a series of microprocessors designed by INMOS in the 1980s for use in parallel computing systems. Each transputer had its own memory and communication ports.

[6] "Occam's razor", from the 14th century English friar William of Occam, is the philosophical principle that "entities must not be multiplied beyond necessity". It is taken as one of the principles of modern scientific thought: "when explain a given phenomenon, one should prefer the explanation that requires fewest assumptions".

CLP

In the 1980s, Constrain Logic Programming languages (CLP) were introduced. These are languages that allow the manipulation of relations over appropriate domains (see Chap. 13). The idea of adding to logic programming a classical mechanism for the solution of constraints was developed independently by three research groups. Colmerauer and his group in Marseille was the first to define a language with constraints in 1982. This language was PROLOG II, an extension of PROLOG which allowed the use of equations and *inequalities* over terms (rational trees, to be precise). In the middle of the 1980s, the language was extended to PROLOG III which allowed generic constraints over strings, booleans and reals (limited to linear equations). At Monash University in Australia, the language Clp(R) was developed, with constraints over reals. Jaffar and Lassez defined the theoretical aspects of the CLP paradigm. It was shown how all the various logic languages with constraints could be seen as specific instances of this paradigm and how the paradigm inherits all the main results from logic programming. Finally, Dincbas, van Hentenryck and others at ECRC in Munich defined Chip, an extension of PROLOG which allowed various types of constraint, in particular constraints over finite domains.

16.6 1990s

The 1990s, as we discussed in Sect. 15.1.2, saw the rise of the Internet and the World Wide Web, two technologies that changed many aspects of computing, including programming languages. New possibilities included connecting millions of computing devices in a network, sharing data and programs that reside on machines separated by thousands of kilometres, transmitting data and accessing saved information using channels shared by thousands of users. All this potential introduced problems of efficiency, reliability, correctness, and security that involve the entire spectrum of levels present in a computer system—from the level of communication protocols to the languages used for the final applications.

From the programming language viewpoint, the most relevant aspect of these years was the definition of the Java language. In the same decade, HTML (HyperText Markup Language) was defined by Tim Berners-Lee in 1989 to specify the content of Web pages. HTML, although important, is not covered in this book because it is not a programming language in the strict sense.

Java

The object-oriented language Java was developed by a group (the *Green team*) led by James Gosling at Sun Microsystems. The original project, started in 1990, aimed to define a language based on a new implementation of C++, that could be used in small, relatively low-power computing devices connected to a network and linked to a television set used as input/output peripheral. These devices were intended to be a kind of *ante litteram* browser, used for purposes similar to modern web browsing, long before all the necessary technology was available. The initial language, first

available in a functioning version in 1992, was pretty much ignored. In 1993, however, the release of Mosaic, the first Internet browser, immediately caught the Green team's attention. They saw that the language they were working on had great potential in the world of the Web. In fact, the low communication bandwidth available to home computers and too many requests to servers for the most popular web pages made early browsers painful to use. These problems could be solved, at least partially, by sending little programs (*applet*s) across the network, so that they execute on the user's client machine when it requests a particular service. In this way, Web pages become more interactive (eliminating round-trips or sensibly reducing their weight on the connection) and the load on the server from which the service is required is reduced. The language to be used to implement such applets would have to satisfy the following requirements:

Portability When a program is sent to a remote machine, the architecture of this machine is not known. It is therefore necessary for each client machine to have an implementation of the applet's language. And this is all the more difficult the more the language is is large and complex.

Security Executing programs received remotely from the network requires precise guarantees about the security and reliability of these programs.

Java was designed by taking into account these two basic requirements, in the context of the object-oriented paradigm.

The first problem was solved by defining the *Java Virtual Machine* (JVM) and its associated bytecode. A Java program is translated (compiled) into an intermediate language—the Java *bytecode*—whose abstract machine is precisely the JVM. This machine, which is much simpler than the entire Java machine, is implemented in an interpretative mode on different physical machines. A Java program, once translated to bytecodes, can therefore be sent over the network to be locally executed on the user's machine. The real applicability of this approach was shown in 1994, when Sun developed the HotJava browser, with full support for Java applets through a JVM included with the browser. It was in 1995 that Java started its rapid expansion, when the JVM was incorporated into the Netscape browser (which, in its turn, was a reincarnation of the Mosaic browser).

The security problem, on the other hand, was addressed by various techniques. First, Java was designed with a type system guaranteeing type safety. The execution of a Java program can not cause a runtime type error that is not detected and signalled by the compiler or the abstract machine (at least for programs written making use only of the sequential subset of the language). Type safety is obtained by type checking at three levels. First, the Java compiler, like those for other statically typed languages, does not permit the translation of programs violating the Java type system. Second, the bytecodes that result from the compilation are also checked by a type checker before execution and, finally, at runtime, the bytecode interpreter performs some type checks which, by their nature, cannot be made statically (for example, array bound checking).

Another important design choice in Java to improve the reliability and simplicity of the language is the avoidance of explicit handling of pointers and the presence of a garbage collector to recover unused memory. Thus, even if any Java object is accessible using (an abstract version of) pointers and memory is dynamically allocated, there is no "pointer" type. Let us recall that in Java values of basic types (numbers, boolean, and characters) are not objects; names declared of these types are true modifiable variables. On the other hand, variables representing objects use the reference model (recall Sect. 6.2.1), and thus a program only handles indirect references to objects, without being allowed to freely manipulate those references. Scoping is static and parameter passing is always by value (where objects are passed, the value passed is a reference to the object, thus realising a parameter passing by object reference, see the box on Sect. 7.1.2).

In addition to dynamic method dispatch, which is typical of object-oriented languages, Java also allows the dynamic loading of classes. During execution, if a program invokes a method on a class that is not present and which, for example, resides in a physically remote location on the network, the class can be dynamically loaded into the Java virtual machine. This incremental approach allows programs to start executing even though some of their components are still missing, something which has been of practical use for browsers.

Finally, as we saw in Sect. 14.7, Java allows multiple threads to run concurrently. Synchronisation and communication primitives form an important part of Java's design and contribute to the portability of the language since they do not refer to the system operations of a specific platform.

All these security and reliability features have a cost in terms of efficiency. The existence of runtime type checks, the bytecode interpreter, the garbage collector, and several other aspects significantly affect the execution time of programs. However, this inefficiency is not particularly important for the application domain typical of Java programs. In particular, in the context of web browsers, the time spent waiting for network communications makes the execution times of Java applets negligible.

The Dynamic Four: Python, JavaScript, PHP, and Ruby

Besides the explosive success of Java, the 1990s saw the introduction of four programming languages which would become some of the most popular ones of the following decades. These are Python (1991), JavaScript (1995), PHP (1995), and Ruby (1995).

The foremost commonality of the Four is that they are dynamically typed, interpreted, *scripting languages*. The category of scripting languages takes its name from the support that these languages provide in letting users quickly specify "scripts"—one can draw a parallel with the namesake written text of a play that describes the actions of actors—that coordinate the interactions among different facilities (resources, instructions, tools) provided by an existing system. For example, if we look at operating systems, examples of scripting languages are the Bourne shell (introduced in 1979) for Unix systems and the batch file language (from the early 1980s) for DOS/Windows. These languages allow the user to compose commands

of the operating system in a structured way (e.g., execute command X, capture its output, compare the latter with some constant and either execute command Y or Z). Since the purpose of scripting languages is to rapidly put together a small application that coordinates other programs (e.g., the commands of the operating system), they are usually interpreted rather than compiled.

Scripting languages are a staple element of vast software environments, such as operating systems. However, they are common in other classes of software applications that require the coordination of a multitude of functionalities, like text editors—e.g., EMACS (1976) is famous for its flexibility thanks to its deep integration with LISP, used as its scripting language—and computer games—Lua, a scripting language that appeared in this decade (1993), can still be found in games developed 30 years after its introduction.

Python, JavaScript, PHP, and Ruby fall into this category of languages, although they present relevant differences regarding their main application contexts.

Python, developed by Guido van Rossum, sees the confluence of two traditions. The first is that of interpreted languages with a simple syntax, intended for teaching (e.g., by replacing scope brackets with indentation, to also enforce consistent code-formatting conventions) and prototyping. The second is that of scripting languages for operating systems. For instance, one of the key points for the development of Python was to ease working with files and common data structures such as lists, associative arrays (called "dictionaries"), and sets through constructs like collection comprehension—a terse syntax to generate collections of elements inspired by the set-builder notation—and generators (see Sect 7.2.2). The simplicity and high modularity of Python played a crucial role in attracting contributions from disparate communities (e.g., data science and scientific computing), which helped to increase the diffusion of the language.

The popularity of the other three languages is linked to the rise of the Web. PHP (originally defined as the acronym for Personal Home Page language) and Ruby became renowned for easing the building of server-side Web pages.

The origins of PHP are linked to the Common Gateway Interface (CGI), a standard that appeared in 1993 to regulate how Web servers can execute external programs. The lack of portability and complexity of using compiled languages for CGI motivated the original author of PHP, Rasmus Lerdorf, to create a language whose interpreter would "live" next to the Web server (deeply integrated with CGI) and whose syntax would make it easy to generate content—e.g., PHP allows users to embed Web-page markup in programs, and it provides built-in facilities to interact with databases.

The motivating origins of Ruby follow those of Python—a simple language for fast prototyping. In particular, its developer, Yukihiro Matsumoto, wanted a scripting language built around foundations that permeated all aspects of the language, so that, by knowing these foundations, users would quickly grasp how the constructs of the language worked. The author called this the "principle of least surprise". For example, one foundation of the language is object orientation, and all values in the language are objects (including primitive values, classes, and types); another is that all instructions are expressions (even statements), which return values, and that all instructions are executed imperatively (even declarations). The language rose in popularity with the

wide adoption of Ruby on Rails, a framework that helped in standardising the way to build and deploy Web applications.

The last of the Four, JavaScript, owes its fame to being one of the first languages able to run in Web browsers—so, we can see it as complementary to the PHP and Ruby server-side programs discussed above. JavaScript is also famous for another record: being one of the prominent misnomers in the history of Computer Science. Indeed, a candid interpretation of the name "JavaScript" hints at an interpreted version of (and for scripting with) Java. The (maybe more interesting) reality is that the name was a clever move by Netscape to exploit the popularity of Java and promote the new client-side scripting language included in their browser. Legend has it that the author of JavaScript, Brendan Eich, had only a few weeks to modify the LISP-like language (Scheme) he was embedding into the browser and make it as syntactically similar to Java as possible. In particular, and differently from Java, JavaScript adopts prototype-based object orientation (cf. the "Classless languages" box on Sect. 10.2.2) and (inherited from its LISP origins) it supports first-class functions. Subsequent runtimes allowed programmers to run JavaScript applications outside Web browsers (e.g., for server-side scripting), which further contributed to increasing the popularity of the language.

The Functional Duo: Haskell and OCaml

The 1990s also saw the introduction of two functional programming languages, Haskell (1990) and OCaml (1996), which, in different ways, contributed to experimenting with, evolving, and disseminating functional programming constructs. They became influential progenitors of many languages introduced in the following decades.

Haskell is a pure, side-effect-free (cf. Chap. 11) functional language introduced by the late 1980s functional-programming community. Haskell aimed to focus the efforts of the community on growing a single, comprehensive programming language and using it to promote the adoption of the functional paradigm. Thus, Haskell was designed to support a broad set of features found in other functional languages of the time. Moreover, it had to be flexible enough to allow researchers to quickly design, implement, and disseminate their ideas within the community. The language also had the objective of being a technology for building real applications, so that it would work as a vehicle to spread the usage of the functional paradigm (for real-world applications, as well as in teaching).

As a consequence, Haskell inherited many, widely-adopted features of pure functional languages that preceded it, among which:

- Being founded on the call-by-need/lazy evaluation strategy (cf. Sect. 11.2.3);
- An advanced type system extensively based on abstract data types (cf. Sect. 9.1) and *algebraic data types*—we saw examples of the latter when we presented tagged unions/sum types (cf. Sect. 8.4.3) and product types/records (cf. Sect. 8.4.1);

- *Type classes*, which, at their basic level, are constructs of the type system introduced to support ad-hoc polymorphism through the definition of constraints on parametrically polymorphic types (cf. Sect.8.7.2);
- The deep integration of pattern matching (cf. Sect. 11.3.3) in the language and its strong synergy with algebraic data types (e.g., to express choices with tagged unions and deconstruct records);
- The (optional) use of indentation to structure declarations for readability and teaching.

The language is famous also for spreading concepts from the functional tradition (as per its stated purpose) like the usage of algebraic data types such as Option (cf. Sect. 8.4.3) and the Either/Maybe types and the introduction of *monads* (since its 1996 incarnation) to handle compositionality and side effects.

The second language of the Duo is OCaml. As its name suggests, the language is an offspring of the Caml language (introduced in the mid-1980s), itself a dialect of the ML family (cf. Sect. 11.8). Both languages are projects developed by several researchers at the French Institute for Research in Computer Science and Automation (INRIA). OCaml owes its prefixing "O" to the prominent addition of an object layer that allows users to mix the functional and object-oriented styles. The language inherited and evolved many traits from the ML tradition, such as imperative control constructs, an ML-like static type system with advanced type inference features, parametric polymorphism, and pattern matching. In addition to its linguistic characteristics, OCaml enjoys the results of decades of research devoted to the improvement of the Caml runtime (of course, optimised to run the constructs of the language), which allowed it to equip an interactive interpreter (for fast prototyping), a byte-code compiler (for portability), and a native-code compiler (for performance). These elements greatly contributed to helping OCaml gain a strong presence in the industry, where the language has been largely used to implement low-level tools for operating systems, to develop Web applications, and (most famously) in the financial sector.

The Libraries for Concurrency

Despite the development of several specific languages for concurrent programming, such as those mentioned in the previous sections, much of current concurrent programming uses a traditional sequential language augmented with appropriate libraries that realize the functionality needed to handle concurrency. Many of these libraries were defined in the 1990s.

In particular, in the mid-1990s, the Pthreads library was defined as part of the POSIX (Portable Operating System Interface for Unix) family of standards. This is a set of C routines that allow a normal C program to be multithreaded using a dozen functions for thread management and synchronisation.

Two widely used packages for concurrent programming with message exchange are Parallel Virtual Machine (PVM) and Message Passing Interface (MPI). Both of these libraries allow processes in a distributed environment to be built using

programs written in a sequential language (typically C or FORTRAN), which then communicate and synchronise by calling PVM or MPI functions. The functionality offered by the two libraries is similar, although PVM allows greater flexibility in handling heterogeneous machines and faults, while MPI supports a greater variety of communication primitives.

Packages for distributed programming that support message exchange and remote method invocation are also available in Java, as we saw at the end of Sect. 14.7 talking about `java.net`, `java.rmi`, and `java.rmi.server`.

In the area of theory, an important contribution of the 1990s was the definition of the π-calculus by Robin Milner, Joachim Parrow, and David Walker. This is a calculus that develops the work begun with CCS by introducing the possibility of describing mobile systems whose configuration varies dynamically. Variations and extensions of the π-calculus are also used to describe biological and economic systems.

16.7 2000s

The 2000s saw the introduction of the service-oriented programming paradigm (cf. Sect. 15.1.4) and of the languages discussed in Chap. 15, dedicated to the description of service interfaces (e.g., WSDL), orchestration (e.g., BPEL) and choreography (e.g., WS-CDL).

Apart from these new paradigm proposals, the 2000s witnessed a sort of slowdown in the appearance of new, popular programming languages. This does not mean that new languages were not introduced in those years, but rather that these proposals struggled to attract the attention of programmers that, at the time, was shared between long-time strongholds, like C and (the slightly more recent) C++, and newcomers like Java, PHP, and Python.

A language that succeeded in gaining some attention is C#. Introduced in 2000 by Microsoft, C# is a language much closer (in features) to Java and C++ than its eponymous father, C. Among their objectives, the designers of C# wanted to promote the adoption of the .NET framework—a technology based on the Common Language Infrastructure which, similar to the Java runtime, supports the compilation of high-level programs into intermediate-language ones, able to run on different platforms and computer architectures. The language is object-oriented, garbage-collected, and type-safe (à la Java) but also supports "unsafe" blocks, where the programmer can directly manage the memory addresses of pointers. Along with C#, in 2005, Microsoft introduced F#, which is a functional language directly influenced by OCaml. Also in this case, the language was intended to marry the desirable linguistic features of its inspirer with the facilities provided by the .NET framework.

We mention this duo of "sharp" languages also because they illustrate a phenomenon, started in this decade, where we consistently see cross-pollination from functional to object-oriented languages—i.e., where the latter adopt concepts and constructs introduced and proved useful by the former. Indeed, while F# has a smaller

userbase than C#, it has been serving as a test bed to introduce new language constructs that eventually reached the more mainstream C#.

A similar phenomenon happened between Java and another popular language introduced in this decade: Scala.

Scala

Work on Scala started in 2001, driven by Martin Odersky. The name "Scala" is the blend of the words "scalable" and "language" and it was given to convey one of the objectives behind the design of the language. Scala shall scale according to the needs of the user. More concretely, the design of the language strived to introduce syntactic facilities that helped users in defining, within the language itself, domain-specific languages.

The idea behind using a domain-specific language (DSL) is that, instead of modelling one's problem to fit the abstractions of a general-purpose language—where the programmer needs to figure out a way to represent the entities of their problem's domain as, e.g., objects and methods—the user can define a DSL where those entities are first-class citizens. The semantics of the DSL defines the way to solve the problem. Examples of famous DSLs are HTML, for Web pages, and TEX (1978) and LATEX (1984), for document preparation. In the case of Scala, the user writes the DSL as a "guest" language embedded within the "host" (Scala), which the developer uses to describe both the DSL's syntax and to implement its semantics.

To achieve this flexibility, Scala integrates concepts and language constructs from object-oriented and functional programming. It is a class-based language where all values are objects. Moreover, it is deeply integrated with *traits* (which the language helped popularise) as a middle ground between interface implementation—where a class must implement a set of method signatures—and multiple inheritance—where a class acquires method implementations from other classes. Scala has also full-fledged functional language features, like first-class, higher-order functions, pattern matching, lazy evaluation, and currying.

In addition to the above design principles, the implementation of Scala was targeted to overcome the perceived limitations of the contemporary version of Java. This motivated the implementation of a compiler for Scala that targets the Java Virtual Machine and, thanks to byte-code compatibility, that allows Scala programs to include and handle Java classes.

In the following years, Java would (indirectly) integrate features first appeared in Scala and tested by its programmers, like advanced type-inference mechanisms, refined generics (cf. Sect.10.4.2) with covariance-contravariance overriding (cf. Sect.10.4.5), functional interfaces, pattern matching, and *sealed classes and interfaces* (similar to traits).

16.8 2010s

The landscape of programming languages in the 2010s witnessed two main trends, which motivated the introduction of new proposals and the evolution of existing ones. On the one hand, we see new languages that try to "dethrone" the position of C. At large, these new proposals aim to allow developers to implement programs that are as performant as their C alternatives (e.g., in speed and memory occupation) but with increased support for correctness and safety. On the other hand, we see proposals that introduce static guarantees on dynamically-typed languages via extensions that modulate the amount of typing information the programmer must provide.

The C Contenders: Go, Rust, and Swift

Since its debut, C gained and subsequently maintained a strong position as the language one would use to implement performance-critical software. Moreover, there is a folklore argument among programmers whereby C is their go-to language thanks to its small set of features and straightforward execution model—as opposed to the larger set of language constructs and the more complex runtime of alternatives, like C++ or Java, which can make it harder to reason on the behaviour of programs. Notwithstanding these good characteristics, any programmer with at least a little experience with C would recognise its many limitations (by modern standards), like error-prone memory-management primitives and minimal support for type-checking and structured error handling.

In this decade, we find several new languages that present themselves as contenders for C, i.e., languages that support programmers in writing correct programs that are also efficient. In the following, we introduce three of the most famous.

Go is the oldest of these contenders and, although missing the 2010s mark by a couple of months (it first appeared in November 2009), we can safely classify it among the languages that characterised the decade. The language was developed within Google by Robert Griesemer, Robert Pike, and Ken Thompson (of C fame). The influence of C on Go relates also to its simplicity. Indeed, besides the close resemblance of Go's syntax to that of C, Go includes some features to further simplify it. For example, the language is structurally typed, it requires minimal annotations from the user thanks to type inference, and it is generally type-safe—although, like e.g., C# and Rust (the latter, discussed below), Go provides instructions to bypass the type system and express low-level logic, e.g., pointer arithmetic. Another feature is that Go provides inbuilt, syntactic support for concurrency through lightweight processes (called "goroutines") that communicate via channels (cf. Sect. 14.3.1) à la CSP/CCS. Another relevant feature that positions Go in the group of C contenders is a compiler that, although employing garbage collection for memory management, produces efficient executables.

Rust, initially developed as a personal project by Graydon Hoare, in 2010, became a project officially sponsored by the employer of Hoare, Mozilla—a free software community founded by members of Netscape. Like Go, also Rust's syntax builds on that of C (with OCaml as a second strong influencer). A distinctive characteristic that

contributed to the popularity of Rust is its usage of types and *lifetimes* of variables to produce efficient programs. Indeed, Rust popularised the usage of an *ownership system* to obtain automatic memory management without the necessity of garbage collectors. Roughly, this is possible because each value in Rust is attached to a variable, which is its exclusive "owner". Owners can transfer the ownership of a value to other variables, e.g., via assignments, and other variables can *borrow* values from their owners. Borrowing means the temporary transfer of ownership, which is returned to the owner after the borrower finished using it. For example, Rust's *borrow-checker* makes sure that the lifetime of the owner of a value extends beyond the lifetime of the borrower of that value. This allows the Rust compiler to perform optimisations, e.g., that once the owner of a value disappears (e.g., by going out of scope) it can safely free any memory in the heap that corresponds to the latter. Borrow-checking also supports the implementation of concurrent programs free from races on data, since memory access always happens in safe conditions, e.g., knowing that a variable has only one owner who can modify it. Rust was also influenced by the functional paradigm, including features like type-class-like traits (à la Haskell), parametric polymorphism, and pattern matching. Like Go and C#, also Rust provides an "unsafe" mode that allows the developer to write low-level code (e.g., where we have multiple pointers that modify the same value in memory) that circumvents the restrictions enforced by the type- and borrow-checker.

Swift (2014) is a programming language developed by Apple, with the purpose to replace Objective-C, a Smalltalk-like object-oriented extension of C introduced in 1984 and used since to program applications for the company's devices. Given this objective, Swift inherited from Objective-C *protocol orientation*, which is a method to implement code compositionality alternative to object-oriented inheritance (which Swift also supports). In protocol orientation, a *protocol* defines a set of fields and operations that one can attach to a structure or a class. The programmer can write *extensions* of the protocol to initialise the fields and define the behaviour related to that protocol for all structures/objects that implement it. To manage memory in presence of references (e.g., when passing objects), Swift adopts a model of reference counting (cf. Sect. 8.11.1). To solve the problem of memory leaks due to uncollectable reference cycles, Swift lets the programmer mark references labelled as strong and weak. Strong references are the ordinary ones (e.g., when assigning an object to a variable), whose creation and deletion impact the count of a reference. Weak references do not modify the reference count but always correspond to Optional types. In doing so, the language achieves two results. On the one hand, it forces the programmer to check for the presence of the value addressed by the reference, preventing null-pointer dereferencing. On the other hand, one can break reference cycles by using strong and weak references in parent-child structures, e.g., where the parent has a weak reference to the child, but the child has a strong one to the parent.

Gradual Typing and TypeScript

In Sect. 16.6, we discussed how the 1990s saw the introduction of a quartet of interpreted, dynamically-typed languages that became and remained some of the most

popular programming languages of the following decades: JavaScript, PHP, Python, and Ruby.

Besides their specific characteristics and typical application areas, an important aspect that contributed to determining the wide adoption of these languages is the support they give to users in rapidly putting together small programs and prototyping ideas. Indeed, thanks to dynamic typing, developers can build the elements that make up their programs along the way, instead of having to specify the type of those elements beforehand. Since typing happens at runtime, programmers retain the guarantee that, if there is an inconsistent usage of the program's elements, the runtime checks would catch them and avoid unexpected behaviours.

As the number of large software projects developed with dynamically-typed languages grew over time, users realised that trading rapid prototyping off static checks was an unfavourable deal. Indeed, as discussed in Sect. 8.2.1, in static typing, we can see a type as a contract that both the provider (the implementor of a value of that type) and the user (the consumer of the value of that type) agreed to follow. When the type checker analyses a program, it points out all instances where either party does not abide by that contract. On the contrary, dynamic typing reports those instances only when the program reaches an erroneous instruction at runtime. Developers can check that their implementations are correct by assembling exhaustive testing cases for the possible combination of elements of their program. However, the larger the codebase, the more tests one needs to write to exhaustively cover all combinations. Essentially, this activity partially defeats the point of adopting dynamically-typed languages for fast development/prototyping; we chose a dynamically-typed language to let us quickly assemble our programs, but we ended up hampering development due to the need for extensive testing.

Recognising the limitations of both static and dynamic typing, researchers introduced a compromise between the two: *gradual typing*. In gradual typing, users can modulate the amount of typing information they provide in their programs, indicating what elements of their programs the interpreter/compiler can statically and dynamically check.

Interestingly, while one can build a new language that supports gradual typing, the technique naturally lends itself to creating gradually-typed extensions of existing languages (either statically- or dynamically-typed).

As expected (given their popularity) all languages in our 1990s quartet have seen either proposals that either syntactically extend them with gradual types or tools that use the existing language constructs (e.g., as annotations or optional declarations) to introduce types and perform static type checking.

The most famous example among gradually-typed language extensions is TypeScript. Introduced in 2012 by Microsoft, the syntax of TypeScript is a strict superset of JavaScript's, with additional constructs for type annotations—JavaScript is also the intended target compilation language of TypeScript. This means that valid JavaScript programs are also valid TypeScript programs (which the TypeScript compiler can statically check to a limited extent since they lack complete typing information).

The design principle behind TypeScript and other gradually-typed languages is to give programmers the freedom to choose when (during the development of a software project) and where (in the project's codebase) they want the flexibility of dynamic typing or the guarantees of static type-checking. A developer can take an existing JavaScript program, add to it type annotations that the TypeScript compiler would use to type-check it, and subsequently compile the program back into executable JavaScript—the compilation process might not produce the JavaScript program we started with, since it can use type annotations to introduce optimisations and/or additional runtime checks. Similarly, one can choose to add new TypeScript modules to an existing JavaScript codebase and/or modulate which parts of the program shall be statically checked, by including type annotations.

16.9 Summary

In this concluding chapter, we have sought to put the most important programming languages introduced up to the present into a historical perspective. In particular, we strived to explain the reasons for various design decisions and the success and failures of the presented proposals.

An adequate treatment of these questions would require a dedicated book, which would not be easy to write because even if it is not difficult to locate the documentation for different languages (even for extinct ones), determining the influences between one proposal and another is often extremely complicated.

Concerning the possible future (sequential) languages, Chaps. 10–15 contain indications about paradigms in which research effort is currently being concentrated. Probably the programming languages of the near future will develop in this context by seeking to improve on the many weak points of current languages (some cues for reflection in this sense are indicated in the text).

There are other contexts from which significant progress can be expected. One, perhaps the most important, is that of concurrent languages. In particular, as mentioned in Chaps. 14 and 15, technologies such as those that emerged for Service-Oriented Programming and Cloud Computing promise important developments in both theory and practice, with far-reaching application spin-offs. Important innovations in languages can also be expected from areas such as bio-informatics and quantum computing. Although Church's thesis seems destined to withstand the stresses coming from these new areas, there is no doubt that biological or quantum computational mechanisms open up hitherto unexplored perspectives. These are, however, developments that will require adequate time: we will probably have to leave it to others, perhaps to one of the young readers of this text, to account for the novelties to come.

16.10 Bibliographical Notes

The bibliography for this chapter is, as can be predicted, endless. Every single programming language has been the subject of numerous publications from manuals to more theoretical treatments. In particular, concurrent and service-oriented programming would deserve a separate bibliography, given the continuously growing number of scholarly publications and educational books devoted to the subject (see also Sects. 14.9 and 15.5). For some main languages currently in use, we note [1] for C, [2,3] for Java, [4] for C++, [5] for ML, and [6] for PROLOG.

Wishing to limit ourselves to publications about programming languages in a historical context, an excellent source of information is the proceedings of the conference on the history of programming languages organised by the American Association for Computing Machinery (see the editions of [7–10]). Almost all the languages mentioned in this chapter are the object of an introductory note in these proceedings, written by the language designers themselves. Also, the Annals of the History of Computing from the IEEE contain much interesting material. A historical account of software of the 1950 and 1960s is contained in [11]. For a history of the notion of types in programming languages, see [12].

There are also books that provide an overview of various languages, for example, [13] and, for older languages, [14]. Some texts, whose purpose is similar to our own, are [15–17]. They also contain short descriptions of the most important languages. In addition, there are various texts which present the history of various aspects of the automatic computer, such as [18]. It can also be interesting to consult monographs about the pioneers of the modern electronic computer because, in addition to biographical aspects, they provide interesting points for reflection on the difficulties encountered by those who, basically, invented a completely new discipline. A good example in this sense is [19].

Finally, to understand the different, often contrasting, viewpoints, of the protagonists of this story of programming languages, it is certainly useful to consult the original articles which they have written. The article by Dijkstra [20] and that by Wirth [21] are two of the many that are available.

References

1. B.W. Kernighan, D.M. Ritchie, *The C Programming Language* (Prentice Hall, 1988)
2. T. Lindholm, F. Yellin. *The Java Virtual Machine Specification*, 2nd ed. (Sun and Addison-Wesley, 1999)
3. J. Gosling, B. Joy, G. Steele, G. Bracha, *The Java Language Specification, 3/E* (Addison Wesley, 2005). The last specification available at the time of printing is that of Java SE 19. docs.oracle.com/javase/specs/. Accessed 14 Feb. 2023
4. B. Stroustrup, *The C++ Programming Language*, 4th edn. (Addison-Wesley Professional, Boston, 2013)
5. R. Milner, M. Tofte, R. Harper, D. MacQueen, *The Definition of Standard ML—Revised* (MIT Press, 1997)
6. L. Sterling, E. Shapiro, *The Art of Prolog* (MIT Press, 1986)

7. L. Wexelblat ed., in *Proceedings of the first ACM SIGPLAN Conference on History of Programming Languages* (ACM Press, 1978)
8. Association for Computing Machinery (ACM), in *Proceedings of the Second ACM SIGPLAN Conference on History of Programming Languages* (ACM Press, 1993)
9. B. Ryder, B. Hailpern eds., in *HOPL III: Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages*, New York, NY, USA (2007). Association for Computing Machinery
10. R.P. Gabriel, G.L. Steele Jr., eds., *Fourth ACM SIGPLAN History of Programming Languages Conference (HOPL IV)*, vol. 4 (Association for Computing Machinery, 2020)
11. T. Haigh, P.E. Ceruzzi, *A New History of Modern Computing* (MIT Press, 2021)
12. S. Martini, Several types of types in programming languages, in F. Gadducci, M. Tavosanis eds., *History and Philosophy of Computing—Third International Conference, HaPoC 2015*, Pisa, Italy, 8–11 Oct. 2015, Revised Selected Papers, volume 487 of *IFIP Advances in Information and Communication Technology*, pp. 216–227 (2015)
13. E. Horowitz, *Programming Languages: A Grand Tour* (Computer Science Press, 1987)
14. J. Sammet, *Programming Languages: History and Fundamentals* (Prentice-Hall, 1969)
15. R. Sethi, *Programming Languages: Concepts and Constructs* (Addison-Wesley, 1996)
16. M.L. Scott. *Programming Language Pragmatics* (Morgan Kaufmann Publishers, 2000)
17. T.W. Pratt, M.V. Zelkowitz, *Programming Languages: Design and Implementation*, 4th ed. (Pearson, 2000)
18. M.R. Williams, *A History of Computing Technology*, 2nd ed. (Wiley, 1997)
19. I.B. Cohen, *Howard Aiken: Portrait of a Computer Pioneer* (The MIT Press, 2000)
20. E.W. Dijkstra, Go to statement considered harmful. Commun. ACM **11**(3), 147–148 (1968)
21. N. Wirth, From programming language design to computer construction. Commun. ACM **28**(2), 159–164 (1985). Turing Award lecture

Index

M. Gabbrielli and S. Martini, *Programming Languages: Principles and Paradigms*, Undergraduate Topics in Computer Science,
https://doi.org/10.1007/978-3-031-34144-1